Henrik Standnes Grønlund

The title of your project thesis should be written here

Any undertitle is written here

Project thesis in Physics and Mathematics
Supervisor: Mats Ernström
Co-supervisor: Douglas S. Seth
December 2026

Norwegian University of Science and Technology
Faculty of Information Technology and Electrical Engineering
Department of Mathematics

NTNU

ABSTRACT

Write an abstract/summary of your thesis, and state your main findings here.

A summary should be included in both English and any second language, if this is applicable, regardless if the thesis is written in English or in your preferred language. These should be on separate pages, the English version first.
Plan for: 29.09

- Se på hvordan ikdv fungerer og lage en generell klasse for å finne symbolene
- Lage en egen LSM solver som tar inn ferdig lagde bølge matriser og finner symbolene
- Lage mer komplekse sammensatte bølger og se hvordan de oppfører seg i sovleren
-

PREFACE

Throughout the text I assume that the reader is familiar with theory within PDE's, numerical methods and the Fourier-space. This is not intended to be a text anyone can pick up, but some of my fellow students working in similar fields should understand it. (Think Vemund Trym (x2) and Magnus)
You may include acknowledgements and thanks as part of your preface on this page, or you may add it as a new chapter after the preface.

CONTENTS

Abstract i

Preface ii

Contents iv

List of Figures iv

List of Tables v

Abbreviations vii

1 Introduction 1
1.1 Motivation 1
1.2 Project description 1
1.2.1 Stakeholders 1

2 Theory 3
2.1 Decay of Fouirier coefficients 3

3 Methods 5
3.1 Method 5
3.1.1 Integrating Factor 5
3.1.2 Exponentital Time Differention 5
3.1.3 Split Step 5

4 Results 7

5 Discussion 9
5.1 Future work 9

6 Conclusions 11

References 13

Appendices: 15

A A - The work flow **16**
A.1 The travelling waves 16
A.1.1 Naive Newton 16
A.1.2 Spectral Newton 16
A.2 Time-dependent KdV 17
A.3 Recovery on KdV data 17
A.4 Full dispersion 18

LIST OF FIGURES

LIST OF TABLES

ABBREVIATIONS

CHAPTER

ONE

INTRODUCTION

Science has long sought to understand the world and how it behaves. To this effect scientist have looked at the world around them and tried to recreate phenomena in controlled environments. This methodology has used the limiting and asymptotic cases to simplify expressions and in doing so found approximate solutions. These simplifying assumptions can give explicit solution forms that only in specific circumstances capture the complexity of the studied problem. This limitation has, in part, been due to simple equipment for monitoring the experiment. Emerging technology can minimize this factor and bridge the gap from theoretical and experimental data.

1.1 Motivation

(Look the the books given by Mats) Wave phenomena are everywhere in emergent reaserch

1.2 Project description

"The goal is to obtain the symbols value by value on the lattice $\kappa\mathbb{Z}$, without approximating them by polynomials. How much of the symbols can be recovered by the data, and for how many solutions, is the question the project is to answer." - Paraphrasing from the project plan

1.2.1 Stakeholders

- Something about how this uses less energy than rivaling methods such as w/SINDy
- Why discovery is of interest

CHAPTER

TWO

THEORY

2.1 Decay of Fouirier coefficients

Intent: Present an explanation for the limiting factors of the recovered symbols.

The decay of the Fourier modes are dependant on the smoothness of the function. For a function in $C^k(\mathcal{S})$ we have that,

$$|\hat{f}(n)| \leq |\frac{1}{n^k} \cdot \int_{\mathcal{S}} f^{(k)}(x) e^{-inx} dx| \tag{2.1}$$

$$\leq \frac{C}{|n|^k} \tag{2.2}$$

For functions in the Schwartz space this definition can be expanded such that

$$|\hat{f}(n)| \leq \frac{C}{(1+|n|)^k} \tag{2.3}$$

$$\leq Ce^{-|n| \cdot k} \tag{2.4}$$

This last result is particularly interesting for us, as this gives exponential decay in the modes, and as such one is limited by the accuracy of the machine for values of n, lets use $k = 2$ and we find that at a given accuracy of 10^{-14} gives $n = 16$

CHAPTER

THREE

METHODS

3.1 Method

In this section the various methods utilized are presented.

3.1.1 Integrating Factor

For the general PDE, $u_t = \mathcal{L}u + \mathcal{N}(u,t)$, multiplication by the exponetial e^{-Lt} reduces the problem to:

$$v_t = e^{-Lt}\mathcal{N}\left(e^{Lt}v\right)$$
$$v = e^{-Lt}u$$

For purely linear problems this method gives an explicit solution

$$u(x,t) = u(x,0)\,e^{Lt}, \tag{3.1}$$

That can be effectivly computed using matrix products.

In the general case with *Nonlinearities* one can use Runge-Kutta or other time-stepping methods. Based on the paper from Kassam and Trefethen [1], ETD methods seems to work quite well.

3.1.2 Exponentital Time Differention

By integrating the IF formula over a *single* time step h we get

$$u_{n+1} = e^{Lh}u_n + e^{Lh}\int_0^h e^{-L\tau}\mathcal{N}\left(u(t_n+\tau), t_n+\tau\right). \tag{3.2}$$

An exact solution! All that is left is to approximate the integral (see Cox and Matthews)

3.1.3 Split Step

Solving a mixed problem is hard, however you can just solve them separate by taking smaller steps. Using the symmetric split step method you get a second

order solution in time, and it it quite simple to implement using the integrating factor for the linear part, and RK4 or some other solver for the nonlinear part. Then one can combine the solutions. Take a half using the linear method, solve the nonlinearity and then take another halfstep.

CHAPTER

FOUR

RESULTS

CHAPTER

FIVE

DISCUSSION

Discuss your results here.

5.1 Future work

Include a section about what should or could be done in future research, or explain any recommended next steps based on the results you got. This should be the last section in the discussion.

CHAPTER

SIX

CONCLUSIONS

Give a concise summary of your research and finding here, and include a short summary of any future work as well.

REFERENCES

[1] Aly-Khan Kassam and LLoyd N. Trefethen. "Fourth-order time-stepping for stiff PDEs". In: *SIAM j. Sci. Comput* 26.4 (2005).

APPENDICES

APPENDIX

A

A - THE WORK FLOW

In this chapter I will recount the work I have done and the steps taken to achieve the final goal.

A.1 The travelling waves

Before one can start solving travelling KdV waves in time one needs to build travelling KdV waves. For me this started with looking at the Airy equation and implementing the $sech^2$ wave, both as a univariale, and a bivariate function. From this I worked on methods of solving them forward in time, and comparing them to their analytic counterparts, however that is a topic for the next chapter. The form of the Airy functions intreged me and I tried to no avail to create my own, the method was unstable numerically and I consiged and used the prebuilt functions from scipy. From this point on, I started looking at newton solvers for the travelling waves.

A.1.1 Naive Newton

Wether the name has any theoretical annotations I do not know, the method I created however was quite naive in its implemntation compared to later once. To start of with I i worked on the physical side, and implemented a newton solver - without jacobian - and found I could create waves of small amplitude, and periods close to 2π. These waves were to the naked eye simmilar to the sine waves but solved the KdV equation and had strict numerical restrictions due to the third derivative that needed to be approximated. These restrictions made me cast a wider net, and I found that the Newton solver also had a Fourier counterpart

A.1.2 Spectral Newton

In the spectral space the equations became sparse, the travelling waves do not contain a wide spectrum, and is real and symmetric. The equations themselves are not that different and teh change was implemented quite quickly. As it would turn out however that the main issue of beeing limited in both amplitude and period persited, and another avenue had to be explored. This came by ways of continuation, by incrementing the amplitude up to the desired size older stable

solutions were built oppon and made larger amplitudes possible. That is one sovles the problem for a small amplitude and use that solution as the initial data for a larger amplitude, until the given amplitude is reached, given it converges.

This concludes the introduction to the travelling waves and the newton solvers.

A.2 Time-dependent KdV

Returning to the prospect of solving PDE's I started by looking into linear problems, and ommiting the nonlinearities. Reading [Trefethen et al.] and some of the contempory methods for solving the full equation, in addition to my own experience with the heat equation and the heat kernel, I found the integrating factor mehod appeling. By solving the equation in this way I "dicovered" that all linear PDE's of this type could be solved by convolution with a kernel. The form of the kernel was given by the linear relation in question. Testing it on the transport equation and heat equation gave the expected results, and I was convinced: The integrating factor method was hot shit.

My dissilutions were eventually cleared up, but not before I created a full solver using both: integrating factor and split-step to work on the linear and nonlinear parts separatly. If I recall correctly the split step failed when I cleared up in a dealiasing issue, now that I have removed the dealiasing I could warrent a revisit. (adding it to the todo) To start of with I, again, looked at the $sech^2$ waves and compared my solution to the analytical one. From here I worked on dealiasing of the nonlinearity - at the time I thought this would be a problem. And I compared both padding and masking, but found no difference in the solver, as masking was the simplest one to implement I continued using this.

New to dealiasing and this *complicated* sovlers, I turned to AI (Claude) for help in how to apply it. The answer given seemed a bit off at the time, but I did'nt know better and added the masking both inside the nonlinearity and outside. Solving it this way my split-step and integrating-factor both worked and were of order 2.

After reading more theory and asking more pointed questions to both my supervisors and AI I changed the way my masking was applied, doing this did not change the results of the split-step method but my IF one failed. Given that creating solvers for the PDE was only a small step in the thesis, I moved on with the SS method.

A.3 Recovery on KdV data

With both a travelling wave generator and a forward solver, I could now start on the recovery process. To get a better feel for the processes I made a solver that only tried to recover the linear symbol. Doing it this way ment I could make one wave and then use LSM on only the unknown part of it. In my first attempt I used the travelling wave assumption and made it work on a static wave profile. The recovered symbol was accurate out to $k = 11$, where - using the assumptions from Theory - the strength of the signal approched the tolerance of the "wave generator". From this platform adding the nonlinearity was a simple procedure, as all I had to do was rework the matrix multiplications.

A.4 Full dispersion

After making speciffic solvers for the KdV equation, as described above. I set out to generalize, trying to capture the respective relations for both Whithams m and the fractional KdV (fKdV). This led me down the path of objective programming. I stared working on a general class that could be initiated as any one of the model problems I wanted to look at, that is I created an abstract sovler class and made a child for each of the equations. Doing it this way I could easily set up test cases and plot the important graphs without having to rewrite or copy code. But the set up phase took some time.

In addition to looking into different equations, I also looked into how to represent the equation in my LSM solver. The main point of contension here was the time derivative. In the article by my supervisors, they estimated it using the Fourier transform in time and only used a few (3 iirc) datapoints in time, though this was also limited by the position of the soliton in the frame. In my own experiments having to refactor the time data so that it is periodic was an issue I didnt find a good solution to. I made individual times for each soliton such that they would move exactly one period in time. This solution however only works on travelling waves where the motion can be periodic in time. Looking at eg. a Gaussian blob this method can not be used.